VR_Week6Demo

February 11, 2026

1 For PF Section

2 Data Visualization Week-6

- BTech Computer Science Stream , Februry 2026
- Week 6 - Data Visualization: context, effective visuals and storytelling
- Name: Vidya Rao
- Reg Number: 12345678
- Date: 12/02/2026

2.1 Previous Lab

In the last session, we had learn a **general workflow** for cleaning tabular data:

1. Load data (CSV → DataFrame)
2. Inspect structure and summary
3. Detect missing values (including placeholders like `-1`)
4. Replace missing values (mean strategy)
5. Detect noisy values / outliers
6. Replace noise values (median strategy)
7. Validate the changes and export results

2.1.1 What is the goal of this demo?

This notebook demonstrates: - How to choose the right visualization - How to interpret plots - How to write a short data story - You will apply the same thinking to MTCARS and CEREALS later.

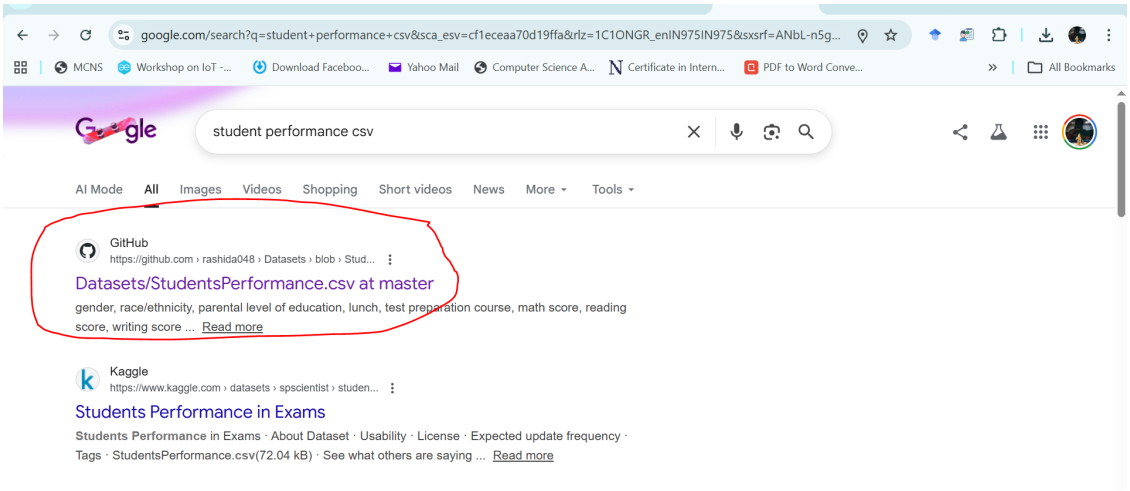
2.2 Step 1: Understand the dataset before plotting

Before creating any graph, we must know: - What does each column represent? - Which columns are numeric? - Which columns represent categories?

2.2.1 Dataset: StudentsPerformance.csv

This dataset contains academic performance details of students along with basic demographic and preparation-related information.

2.2.2 https://github.com/rashida048/Datasets/blob/master/StudentsPerformance.csv



[11]:
```
import pandas as pd

df = pd.read_csv("StudentsPerformance.csv")

df.head()
```

[11]:

	gender	race/ethnicity	parental level of education	lunch	test preparation course	math score	reading score	writing score
0	female	group B	bachelor's degree	standard	none	72	72	74
1	female	group C	some college	standard	completed	69	90	88
2	female	group B	master's degree	standard	none	90	95	93
3	male	group A	associate's degree	free/reduced	none	47	57	44
4	male	group C	some college	standard	none	76	78	75

2.2.3 Mini Skill: Selecting rows/columns using `.loc` and `.iloc`

Before plotting, we often need to **select specific columns** or **filter specific rows**.

- `.loc[]` is **label-based**: use it when you know **column names** or apply **conditions**.
- `.iloc[]` is **position-based**: use it when you want to select by **row/column number**.

[12]:
```
# .loc: select specific columns by name
df.loc[:, ['math score', 'reading score']].head()

# .loc: filter rows using a condition
```

```
df.loc[df['test preparation course'] == 'completed', ['math score', 'reading␣
 ↪score']].head()

# .iloc: select first 5 rows
df.iloc[0:5, :]

# .iloc: select first 5 rows and last 3 columns (scores)
df.iloc[0:5, -3:]
```

[12]:

	math score	reading score	writing score
0	72	72	74
1	69	90	88
2	90	95	93
3	47	57	44
4	76	78	75

2.3 Step 2: Understanding structure of the Data:

- Which columns are numeric (can be plotted)
- Which columns are categorical (used for grouping)
- Whether the dataset is clean and complete

[13]:
```
df.shape
```

[13]: (1000, 8)

[14]:
```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1000 entries, 0 to 999
Data columns (total 8 columns):
 #   Column                       Non-Null Count  Dtype
---  ------                       --------------  -----
 0   gender                       1000 non-null   object
 1   race/ethnicity               1000 non-null   object
 2   parental level of education  1000 non-null   object
 3   lunch                        1000 non-null   object
 4   test preparation course      1000 non-null   object
 5   math score                   1000 non-null   int64
 6   reading score                1000 non-null   int64
 7   writing score                1000 non-null   int64
dtypes: int64(3), object(5)
memory usage: 62.6+ KB
```

2.3.1 Handling Missing Values (Why it matters for visualization)

Even a few missing values can: - distort summary statistics (mean/median) - break plots or create misleading visuals

This dataset is clean, but we will **simulate** missing values to learn the workflow.

[15]:
```
import numpy as np

df_miss = df.copy()
df_miss.loc[0, 'math score'] = np.nan
df_miss.loc[5, 'reading score'] = np.nan

df_miss.isna().sum()
```

[15]:
```
gender                       0
race/ethnicity               0
parental level of education    0
lunch                        0
test preparation course      0
math score                   1
reading score                1
writing score                0
dtype: int64
```

[16]:
```
# drop rows with missing values
df_drop = df_miss.dropna()
print("Before dropna:", df_miss.shape)
print("After dropna :", df_drop.shape)

#Always check how many rows you lost; if too many, we need other strategies
```

```
Before dropna: (1000, 8)
After dropna : (998, 8)
```

2.4 Step 3: Summarize the Data Before Plotting

Visualization without summary is risky.

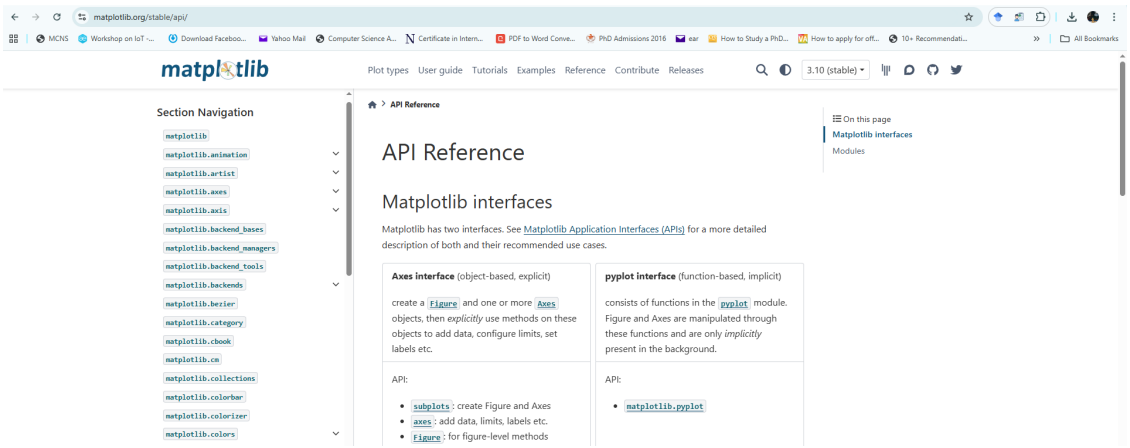
Summary statistics give a numerical overview of: - Typical values - Spread of data - Minimum and maximum values

[17]:
```
df[['math score', 'reading score', 'writing score']].describe()
```

[17]:

	math score	reading score	writing score
count	1000.00000	1000.000000	1000.000000
mean	66.08900	69.169000	68.054000
std	15.16308	14.600192	15.195657
min	0.00000	17.000000	10.000000
25%	57.00000	59.000000	57.750000
50%	66.00000	70.000000	69.000000
75%	77.00000	79.000000	79.000000
max	100.00000	100.000000	100.000000

3 Matplotlib reference document



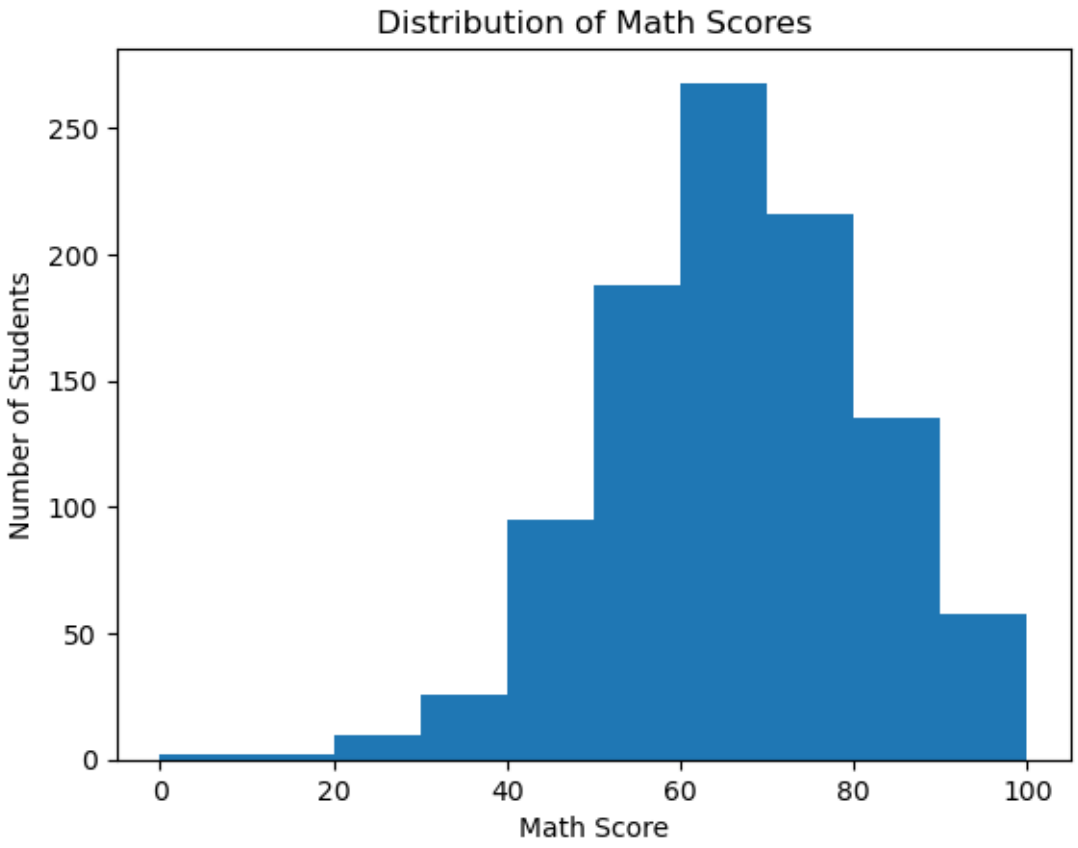
3.1 Step 4: Visualizing Distribution (Histogram)

A histogram is used when: - We want to see frequency - We want to understand how values are spread

Question: **How are student math scores distributed?**

[18]:
```
import matplotlib.pyplot as plt

plt.hist(df['math score'], bins=10)
plt.xlabel("Math Score")
plt.ylabel("Number of Students")
plt.title("Distribution of Math Scores")
plt.show()
```



Observe: - Are most scores clustered? - Are there very low or very high scores? - Is the distribution balanced or skewed?

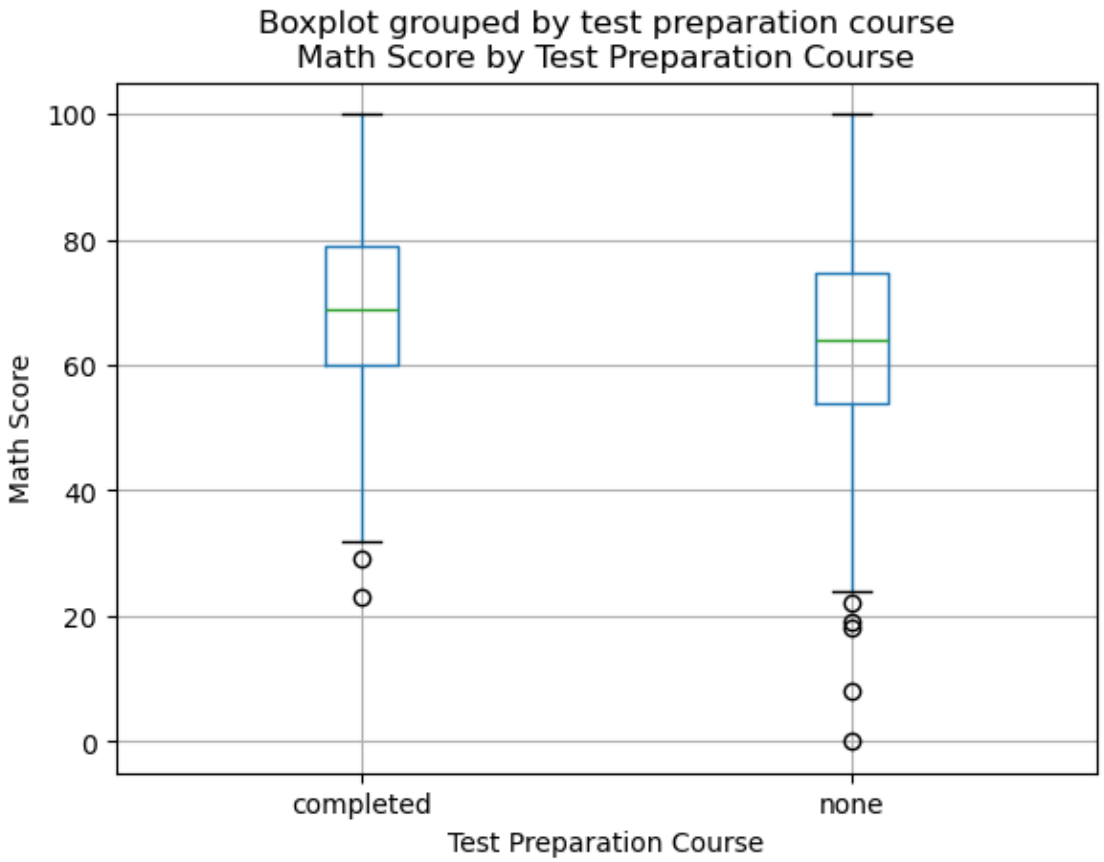
3.2 Step 5: Comparing Groups (Box Plot)

Question Being Explored

Does completing the test preparation course affect math scores?

Box plots are useful for: - Comparing distributions across groups - Identifying medians and outliers

[19]:
```
df.boxplot(column='math score', by='test preparation course')
plt.title("Math Score by Test Preparation Course")
#plt.suptitle("")
plt.xlabel("Test Preparation Course")
plt.ylabel("Math Score")
plt.show()
```



3.2.1 Interpretation Guide

- An upward trend indicates a positive relationship
- Points close to a line suggest strong association

3.2.2 Detecting Outliers using IQR (Why it matters)

Box plots show potential outliers visually, but in data analysis we also need a **rule-based method** to identify them.

IQR method helps us compute an acceptable range: - Q1 = 25th percentile - Q3 = 75th percentile - IQR = Q3 - Q1 - Outliers lie below (Q1 - 1.5×IQR) or above (Q3 + 1.5×IQR)

We will detect outliers for `math score`.

[20]:
```
Q1 = df['math score'].quantile(0.25)
Q3 = df['math score'].quantile(0.75)
IQR = Q3 - Q1

lower = Q1 - 1.5 * IQR
upper = Q3 + 1.5 * IQR
```

```
outliers = df[(df['math score'] < lower) | (df['math score'] > upper)]
print("Outlier count:", outliers.shape[0])
outliers[['math score', 'reading score', 'writing score']].head()
```

```
Outlier count: 8
```

[20]:

	math score	reading score	writing score
17	18	32	28
59	0	17	10
145	22	39	33
338	24	38	27
466	26	31	38

[21]:
```
df_no_out = df[(df['math score'] >= lower) & (df['math score'] <= upper)]
print("Original:", df.shape, "After removing outliers:", df_no_out.shape)

#We do not remove outliers automatically; we only flag them and decide with␣
 ↪justification
```

```
Original: (1000, 8) After removing outliers: (992, 8)
```

3.2.3 Optional

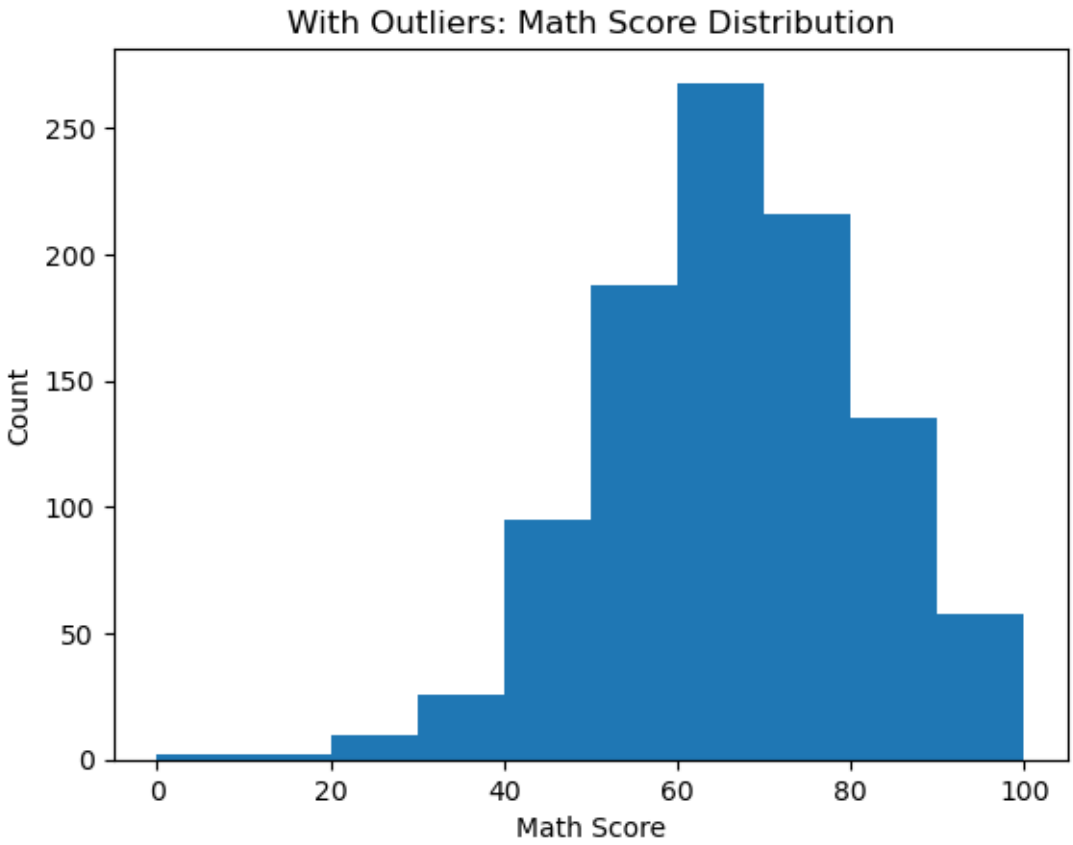
3.2.4 Quick check: Does removing outliers change the story?

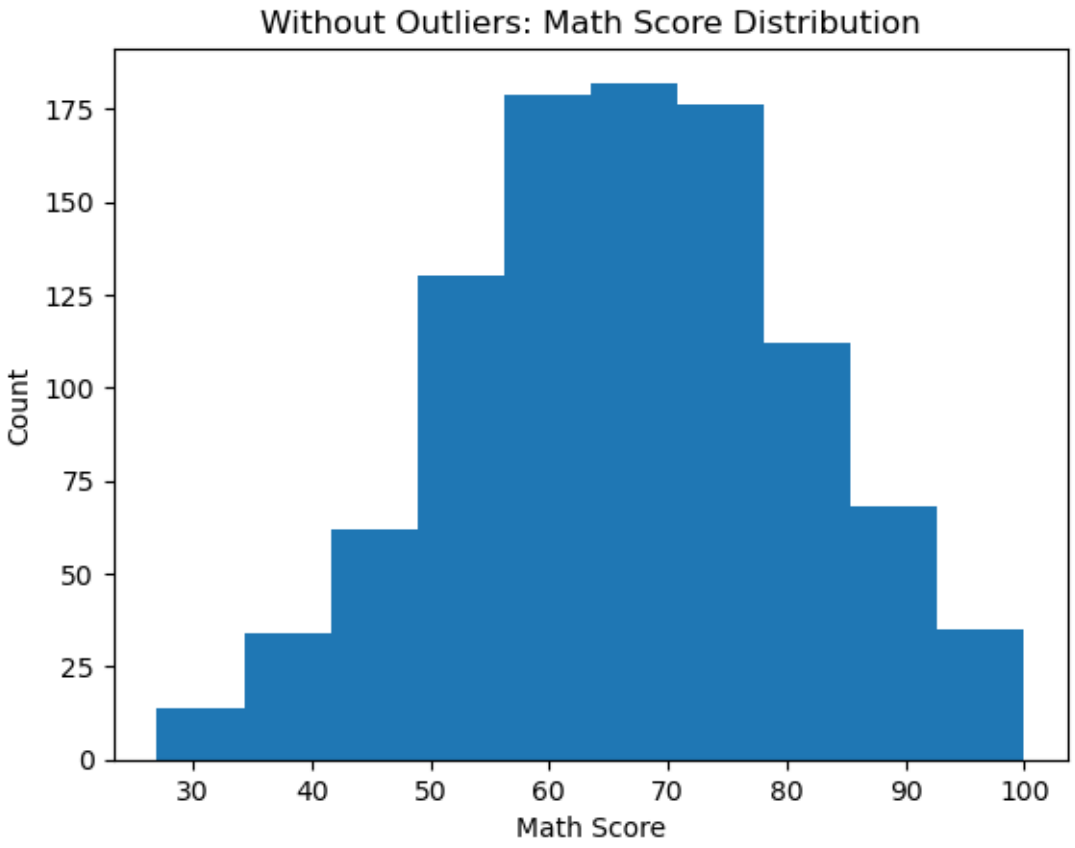
We compare the distribution with and without outliers to see whether outliers are influencing interpretation.

[22]:
```
import matplotlib.pyplot as plt

plt.hist(df['math score'], bins=10)
plt.title("With Outliers: Math Score Distribution")
plt.xlabel("Math Score"); plt.ylabel("Count")
plt.show()

plt.hist(df_no_out['math score'], bins=10)
plt.title("Without Outliers: Math Score Distribution")
plt.xlabel("Math Score"); plt.ylabel("Count")
plt.show()
```


With Outliers: Math Score Distribution
Count
Math Score
0
50
100
150
200
250
0
20
40
60
80
100



3.3 Step 6: Exploring Relationships (Scatter Plot)

Question:

Is there a relationship between reading score and writing score?

Scatter plots are used when: - Studying relationships between two numeric variables - Looking for trends or correlations

[23]:
```
plt.scatter(df['reading score'], df['writing score'])
plt.xlabel("Reading Score")
plt.ylabel("Writing Score")
plt.title("Reading Score vs Writing Score")
plt.show()
```

3.4 Step 7: Writing a Short Data Story

Visualization is incomplete without interpretation.

A data story connects: - The plot - The insight -The implication

Story Structure

1. What do you observe?
2. Why might this pattern exist?
3. What does it imply?

Example

Students who completed the test preparation course show a higher median math score.

This suggests structured preparation helps improve performance.

However, overlap between groups indicates preparation is not the only influencing factor.